```
# Running:

F
Failure:
TestRoman#test_simple [code/unittesting/test_romanbug2.rb:7]:
Expected: "ii"
  Actual: "i"

rails test code/unittesting/test_romanbug2.rb:5


Finished in 0.000466s, 2145.9228 runs/s, 4291.8455 assertions/s.
1 runs, 2 assertions, 1 failures, 0 errors, 0 skips
```

Uh-oh! The second assertion failed. The error message uses the fact that the assert knows both the expected and actual values: it expected to get "ii" but instead got "i." Looking at our code, you can see a clear bug in to_s. If the count after dividing by the factor is greater than zero, then we should output that many Roman digits. The existing code outputs just one. The fix is easy, change the line roman << code unless count.zero? to roman << (code * count):

unittesting/roman3.rb

```
def to_s
  value = @value
  roman = ""
  FACTORS.each do |code, factor|
    count, value = value.divmod(factor)
➤   roman << (code * count)
  end
  roman
end
```

Now let's run our tests again:

unittesting/test_roman3.rb

```
require_relative "roman3"
require "minitest/autorun"
class TestRoman < Minitest::Test
  def test_simple
    assert_equal("i", Roman.new(1).to_s)
    assert_equal("ii", Roman.new(2).to_s)
    assert_equal("iii", Roman.new(3).to_s)
    assert_equal("iv", Roman.new(4).to_s)
    assert_equal("ix", Roman.new(9).to_s)
  end
end
```

produces:

```
Run options: --seed 17659
# Running:

.
Finished in 0.000333s, 3003.0030 runs/s, 15015.0149 assertions/s.
1 runs, 5 assertions, 0 failures, 0 errors, 0 skips
```

Looking good. You can see there's some duplication in the test, and you might be tempted to address it by running each expected and actual value pair in a loop. We recommend avoiding loops in tests, they are often hard to read, and hard to debug if the tests fail. Instead,